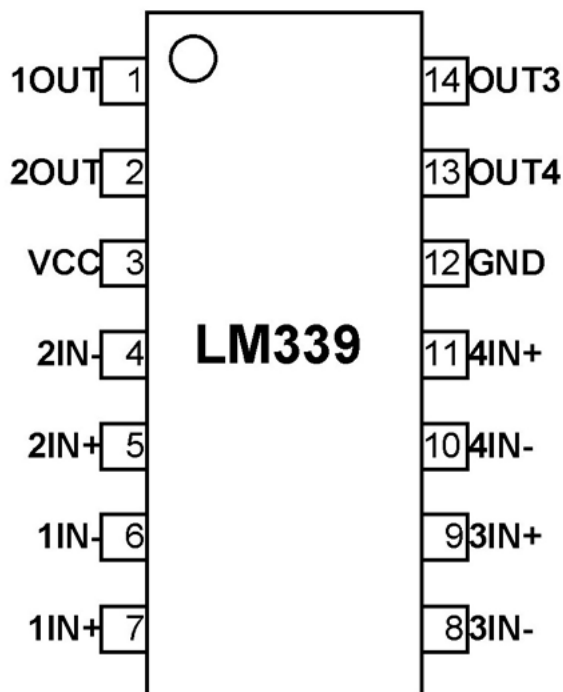


바닥 타일 파이 구성

4개의 압력 센서, 4개의 포텐셔미터, 1개의 비교기(LM339), 4개의 저항으로 구성되어 있습니다. 4개의 포텐셔미터는 비교기의 반전 입력 (IN-)으로 연결되어 비교기가 감지할 압력의 임계치를 설정하는 역할을 수행합니다. 4개의 압력 센서는 각각의 포텐셔미터와 쌍을 이뤄 비교기의 비반전 입력 (IN+)으로 연결되어 실제 상황을 모니터링하며 비교기에 입력값을 제공합니다. 비교기는 포텐셔미터로 설정된 기준값과 압력 센서가 내보내는 압값을 비교하며 OUT핀에서 gpio핀으로 인터럽트를 발생시킬지를 결정합니다. 압력 센서가 내보내는 입력은 노이즈에 취약하여 잘못된 값을 쉽게 반환하기 때문에 풀다운 저항을 추가로 연결하여 노이즈를 최소화하였습니다. 출력핀에도 물리적인 풀업 저항을 연결할 필요가 있으나, 브레드보드 구성이 지나치게 복잡해지고 유지보수하기 힘들어질 수도 있다는 문제점을 고려하여 pigpio 라이브러리에서 제공하는 gpioSetPullUpDown 함수를 이용해 소프트웨어적으로 처리하였습니다. 비반전 입력핀에 압력 센서가 아니더라도 다른 아날로그 센서가 연결되더라도 동작하도록 설계되어 있으며, 원래대로였다면 추가적인 adc를 이용해 처리해야 했을 압력 입력과 비교, 그리고 인터럽트를 발생시키는 동작을 비교기 단에서 전부 처리할 수 있게 되었습니다. 이를 통해 소프트웨어를 간소화하고 cpu의 연산부담을 경감하였습니다. 추가적으로 사용하는 센서의 종류와 관계없이 회로를 재사용할 수 있게 하였습니다.




```
typedef struct
{
    const int in;
    const int out;
    volatile bool isPressed;
    volatile uint32_t lastInterruptedTime;
} PressureTile;
```

각 핀에 필요한 정보를 저장하는 구조체입니다. 비교기의 출력이 연결되는 in, 서버 파이로 인터럽트를 발생시키는 out, 현재 상태를 저장하는 isPressed, 디바운싱 처리를 위한 lastInterruptedTime 변수로 구성되어 있습니다. 이 중 isPressed, lastInterruptedTime 변수는 런타임 중 인터럽트 핸들러로 인해 값이 비동기적으로 변경될 수 있어 컴파일러의 최적화를 방지하고 항상 메모리로부터 값을 읽어올 수 있도록 volatile 한정자를 사용하여 선언하였습니다.

```
void interruptCallback(int gpio, int level, uint32_t tick)
{
    PressureTile* this = NULL;

    for(int i = 0; i < NUM_TILES; i++)
    {
        if(pressureTiles[i].in == gpio)
        {
            this = &pressureTiles[i];
            break;
        }
    }

    if(!this) return;

    if (level == 2 || this->isPressed == level || this->lastInterruptedTime - tick < 10000) return;

    this->lastInterruptedTime = tick;
    this->isPressed = level;
    gpioWrite(this->out, this->isPressed);
}
```

인터럽트 핸들러입니다. This 포인터로 인터럽트가 발생한 핀의 포인터를 받아 처리합니다. 핀의 변화가 없거나, 현재 핀의 상태와 isPressed 변수가 같은 경우 오류가 발생한 것으로 간주하여 처리하며, 추가적으로 0.01초 미만의 연속된 입력은 디바운싱 처리합니다. 위의 케이스에 해당하지 않는 경우 구조체 멤버를 업데이트하고 out 핀의 상태를 조작하여 서버 파이로 인터럽트를 발생시킵니다.

```

int main(int argc, char **argv)
{
    if (gpioInitialise() < 0)
    {
        fprintf(stderr, "pigpio initialization failed\n");
        return -1;
    }

    for (int i = 0; i < NUM_TILES; ++i)
    {
        gpioSetMode(pressureTiles[i].in, PI_INPUT);
        pressureTiles[i].lastInterruptedTime = gpioTick();
        gpioSetMode(pressureTiles[i].out, PI_OUTPUT);
        gpioSetPullUpDown(pressureTiles[i].in, PI_PUD_UP);
        gpioSetPullUpDown(pressureTiles[i].out, PI_PUD_DOWN);
        gpioSetAlertFunc(pressureTiles[i].in, interruptCallback);
    }

    while(1) pause();

    gpioTerminate();
    return 0;
}

```

Main 함수입니다. Pigpio 라이브러리와 핀의 상태를 초기화하고 인터럽트 핸들러를 등록합니다. 추가적으로, 계속해서 입력을 감지할 수 있도록 프로그램을 유지합니다. 위의 회로도 설명에서 간략하게 언급했던 비교기의 출력핀에 소프트웨어적인 풀업 저항을 설정하며, 추가적으로 out 핀에도 소프트웨어적인 풀다운 저항을 설정해 안정적으로 인터럽트를 발생시킬 수 있도록 구성했습니다.